

10 回归

10.2 多元回归

从二元线性回归到多元线性回归——用正交投影、格拉姆矩阵、QR分解统一求解回归系数



鸢尾花书·《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1 掌握二元与多元线性回归模型的代数结构

2 几何视角理解回归平面与超平面如何表示变量间关系

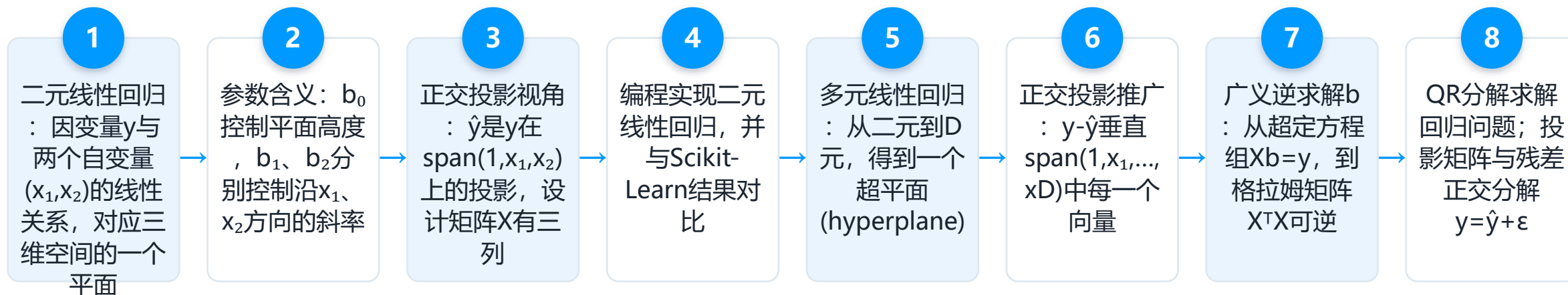
3 设计矩阵的结构，包含常数列与各个特征列

4 正交投影角度理解多元回归

5 最小二乘拟合的原理

6 使用QR分解求解回归问题

从二元回归的平面，到多元回归的超平面



本章每个回归话题，建议从这几方面看

- ① 几何视角：预测值是真实值向量在解释变量张成空间中的正交投影
- ② 代数视角：设计矩阵 $\mathbf{X}$ 、格拉姆矩阵 $\mathbf{X}^T \mathbf{X}$ 、广义逆 $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$
- ③ 最优化视角：最小二乘——最小化残差平方和 $\|\boldsymbol{\epsilon}\|^2$
- ④ 统计视角：残差、拟合优度、显著性检验，留给《概率统计不难》深入讨论
- ⑤ 推广路径：从一元到二元、多元线性回归，再到非线性的多项式回归
- ⑥ 编程视角：手写矩阵公式与调用Scikit-Learn，结果完全一致
- ⑦ 应用场景：预测、趋势拟合、变量关系解释，是数据分析最基础的工具之一

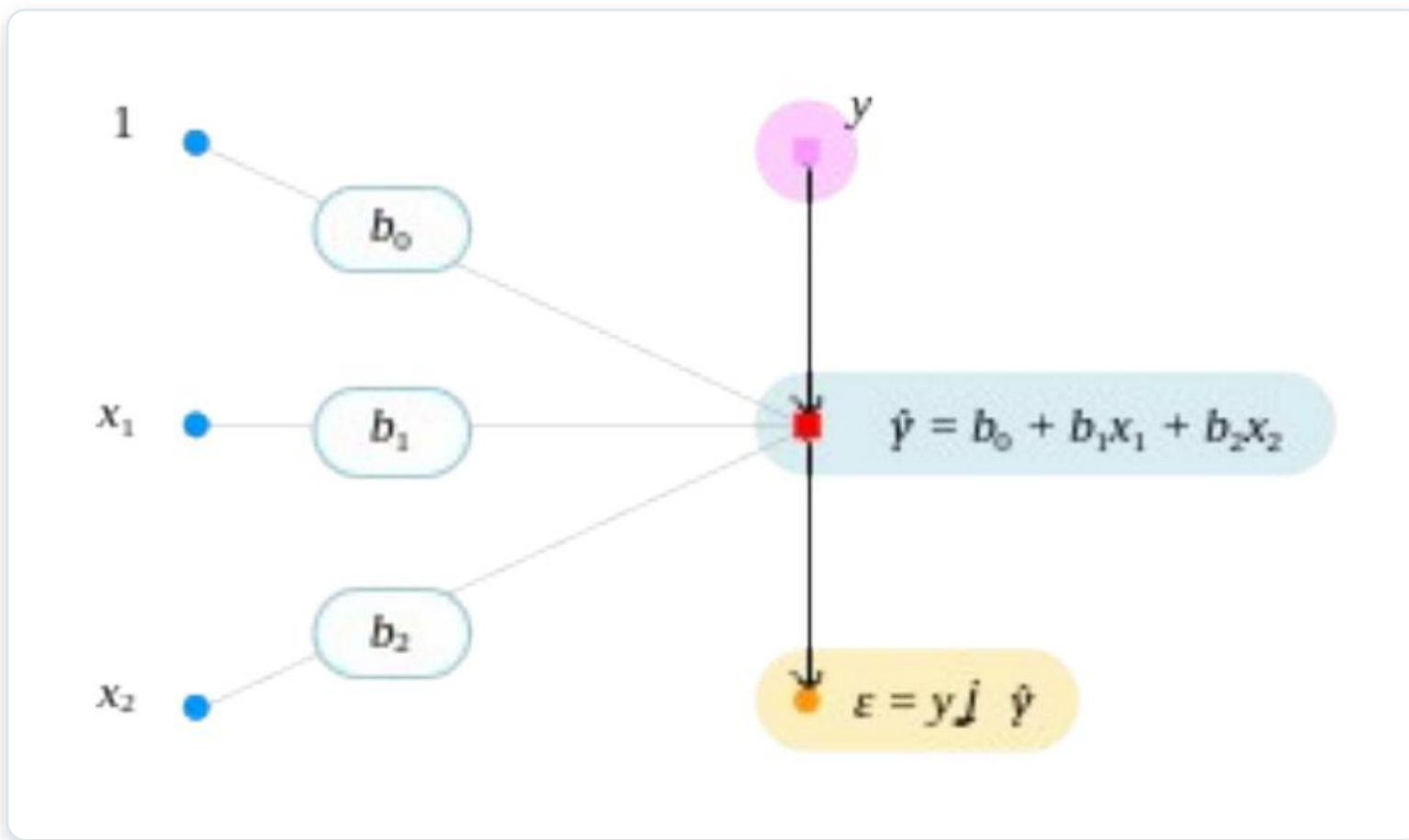
二元线性回归建立 y 与两个自变量 (x_1, x_2) 之间的线性关系

$$y = b_0 + b_1x_1 + b_2x_2 + \varepsilon$$

- b_0 为截距项, b_1 、 b_2 代表自变量系数, ε 为残差项
- 非常建议大家平行阅读本节和上一节10.1——从一元回归升维到二元、多元回归

图1 • 二元线性回归变量关系

- 常数1、自变量 x_1 、 x_2 分别乘以系数 b_0 、 b_1 、 b_2 ，加和得到预测值 $\hat{y}$ ，与真实值 y 相减得到残差 ε



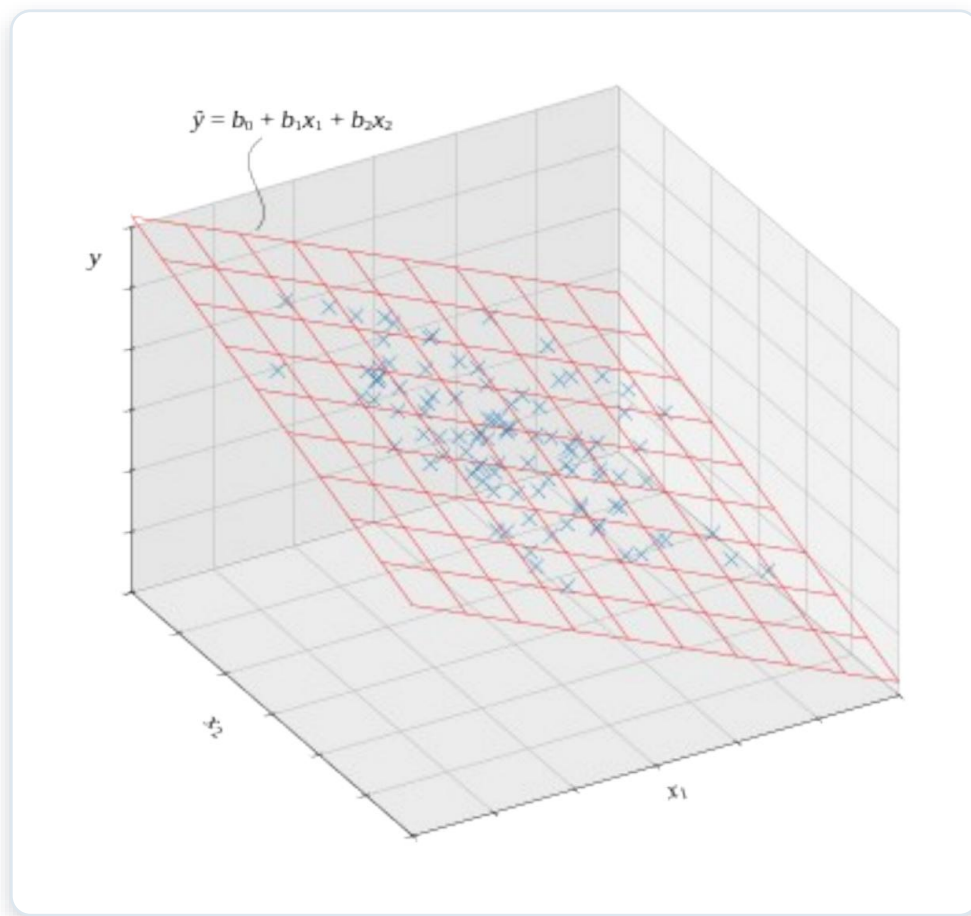
几何角度看，二元线性回归得到三维空间中的一个平面

$$\hat{y} = b_0 + b_1x_1 + b_2x_2$$

- 二元线性回归就是找到合适的 b_0 、 b_1 、 b_2 ，让平面尽可能贴合数据散点

图2 • 二元线性回归对应的平面

- 红色网格面是回归平面 $\hat{y} = b_0 + b_1x_1 + b_2x_2$ ，蓝色叉号是训练数据散点



参数 b_0 、 b_1 、 b_2 如何影响 $\hat{y}=b_0+b_1x_1+b_2x_2$ 平面？

b_0 截距项，控制平面整体高度，即竖直方向上移或下移，但不改变平面的倾斜方向

b_1 绝对值控制 x_1 方向上的斜率，正负决定了 x_1 增大时平面上升还是下降

b_2 绝对值控制 x_2 方向上的斜率，正负决定了 x_2 增大时平面是上升还是下降

图3 • b_0 控制平面的整体高度, $b_1=0$, $b_2=0$

- b_0 从小变大, 平面整体沿竖直方向上移, 倾斜方向不变

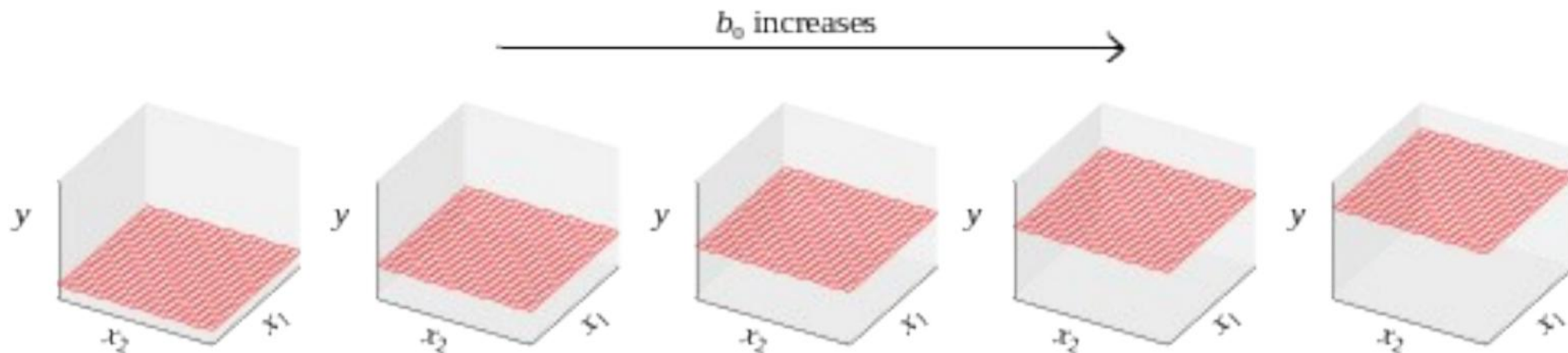


图 3. b_0 控制 $\hat{y} = b_0 + b_1x_1 + b_2x_2$ 平面的整体高度, $b_1 = 0$, $b_2 = 0$

图4 • b_1 控制平面沿 x_1 方向的斜率, $b_0=0$, $b_2=0$

- b_1 从负变正, 平面绕 x_2 轴翻转, 斜率大小和方向都随之改变

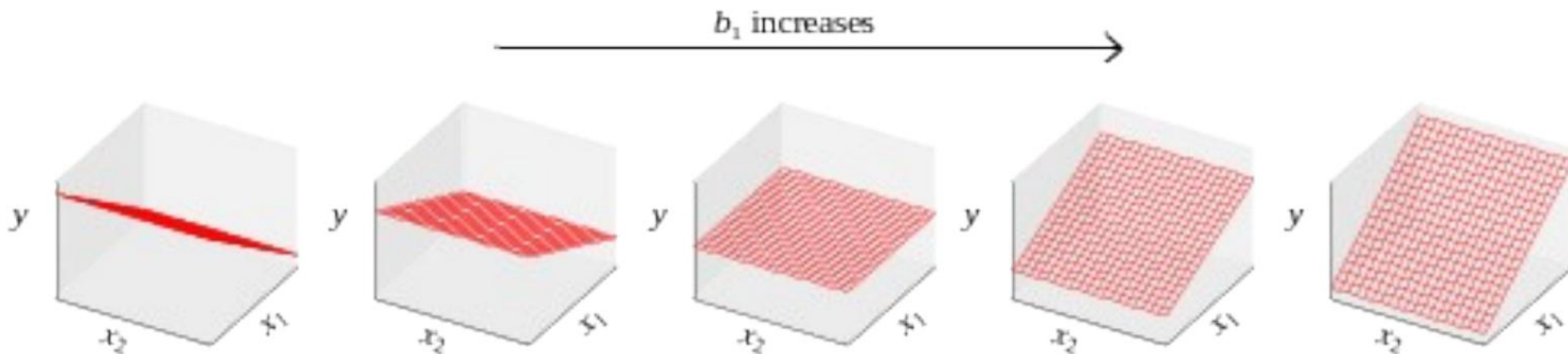


图 4. b_1 控制 $\hat{y} = b_0 + b_1x_1 + b_2x_2$ 平面沿 x_1 方向的斜率, $b_0 = 0$, $b_2 = 0$

图5 • b_2 控制平面沿 x_2 方向的斜率, $b_0=0$, $b_1=0$

- b_2 从负变正, 平面绕 x_1 轴翻转, 斜率大小和方向都随之改变

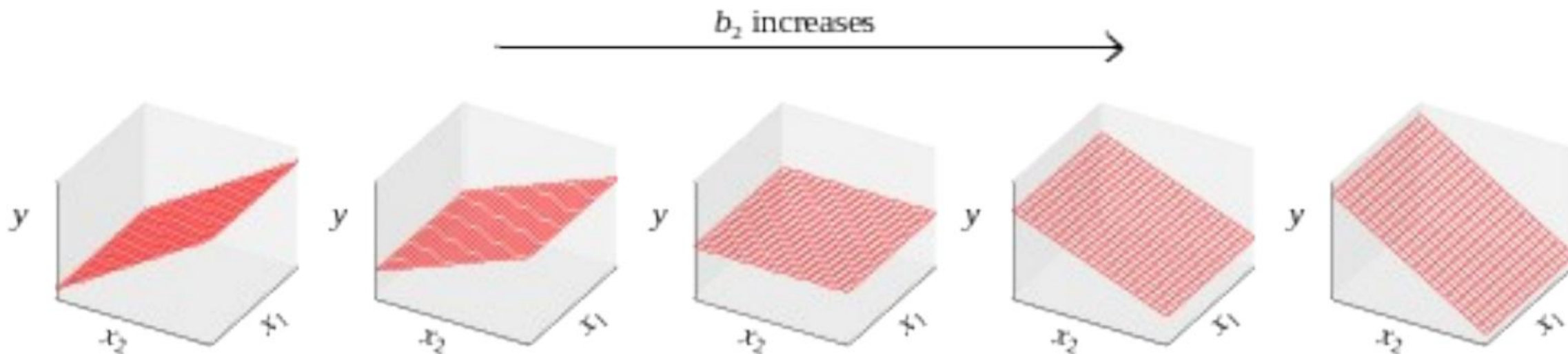


图 5. b_2 控制 $\hat{y} = b_0 + b_1x_1 + b_2x_2$ 平面沿 x_2 方向的斜率, $b_0 = 0$, $b_1 = 0$

三个平面叠加

图6 • 三个平面叠加得到 $\hat{y} = b_0 + b_1x_1 + b_2x_2$

- $\hat{y} = b_0 + b_1x_1 + b_2x_2$ 可以看作三个平面 a) $\hat{y} = b_0$; b) $\hat{y} = b_1x_1$; c) $\hat{y} = b_2x_2$ 的叠加

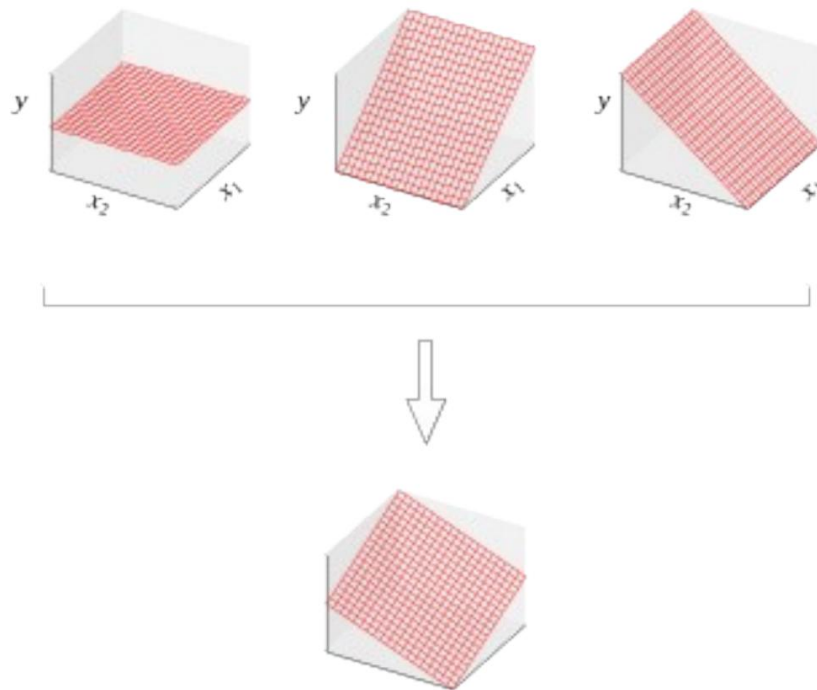


图 6. 三个平面叠加得到 $\hat{y} = b_0 + b_1x_1 + b_2x_2$

真实值列向量 y ，可以写成预测值与残差之和

$$y = \hat{y} + \varepsilon = b_0 \mathbf{1} + b_1 \mathbf{x}_1 + b_2 \mathbf{x}_2 + \varepsilon$$

- $\hat{y}$ 是 y 在 $\mathbf{1}$ 、 $\mathbf{x}_1$ 、 $\mathbf{x}_2$ 张成的平面 $\text{span}(\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2)$ 中的投影

图7 • y 朝 $\text{span}(1, x_1, x_2)$ 投影，二元线性回归

- $\hat{y}$ 是1、 x_1 、 x_2 三个向量的线性组合，误差 $\varepsilon = y - \hat{y}$ 垂直于该平面

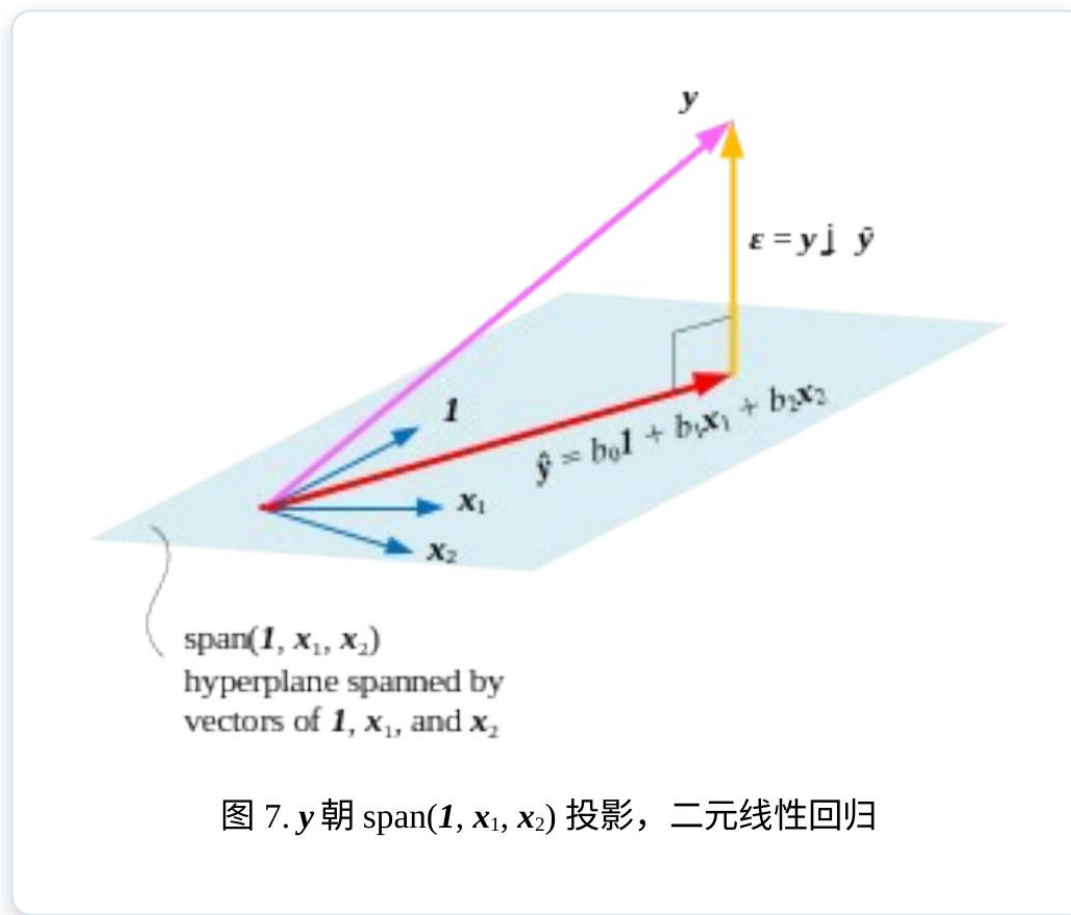


图 7. y 朝 $\text{span}(\mathbf{1}, x_1, x_2)$ 投影，二元线性回归

二元线性回归有两个自变量，设计矩阵X有三个列向量

$$X = [1 \ x_1 \ x_2]$$

- 默认1、 x_1 、 x_2 线性无关，这样X列满秩，格拉姆矩阵 $X^T X$ 满秩，因此 $X^T X$ 可逆
- 实践中列向量也可能线性相关，这种线性回归需要特别处理

图8 • 设计矩阵 X ，二元线性回归

- 把全1列向量与 x_1 、 x_2 两列自变量数据横向拼接，得到 $n \times 3$ 的设计矩阵 X

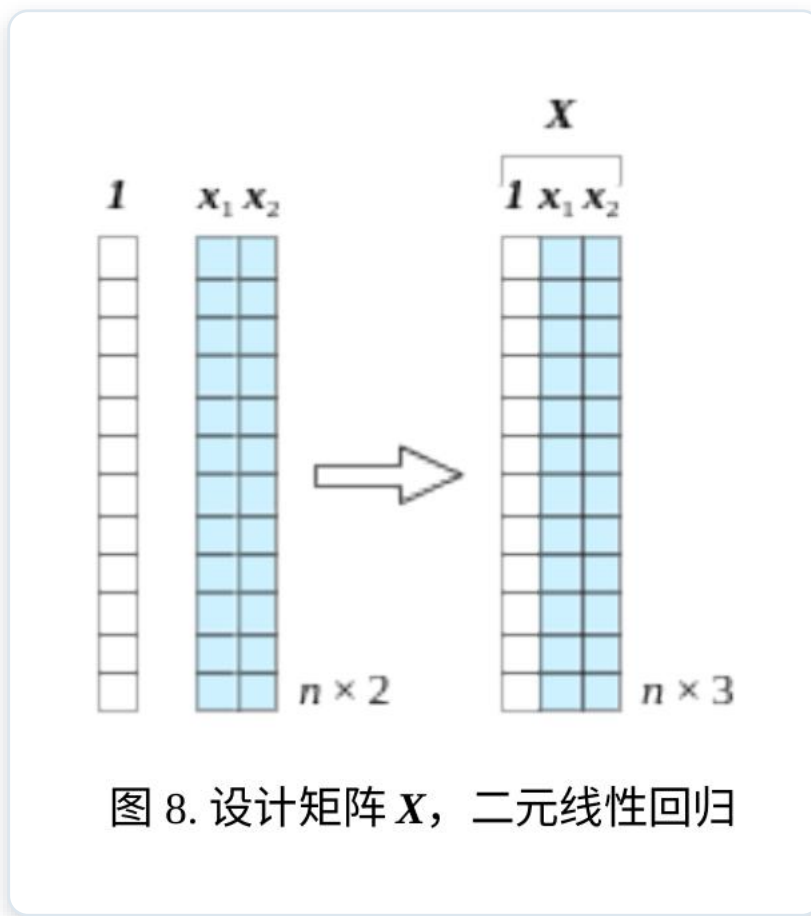


图 8. 设计矩阵 X ，二元线性回归

写成矩阵乘法形式

把线性组合写成矩阵乘法, $\hat{y} = Xb$

$$\hat{y} = [1 \ x_1 \ x_2] [b_0; b_1; b_2] = Xb$$

- b 是由三个回归系数 b_0 、 b_1 、 b_2 堆叠而成的列向量

图9 • 矩阵运算得到 $\hat{y}$ ，二元线性回归

- 设计矩阵 X 与系数向量 b 做矩阵乘法，直接得到预测值列向量 $\hat{y}$

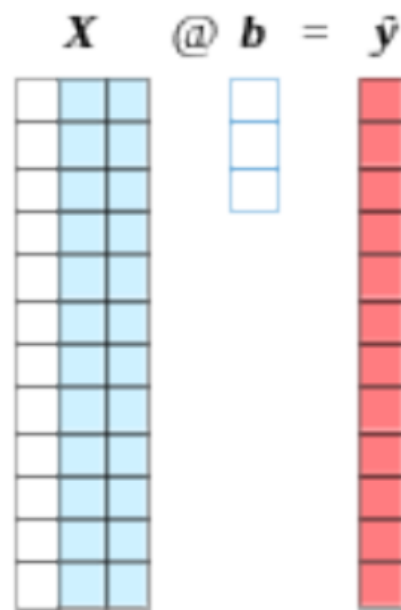
$$X @ b = \hat{y}$$


图 9. 矩阵运算得到 $\hat{y}$ ，二元线性回归

利用格拉姆矩阵与广义逆，求解回归系数b

$$\mathbf{b} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- $\mathbf{X}^T \mathbf{X}$ 是X的格拉姆矩阵；先算格拉姆矩阵及其逆，再乘以 $\mathbf{X}^T$ ，得到广义逆 $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$
- 广义逆 $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ 与真实值向量 $\mathbf{y}$ 相乘，就得到最优参数向量 $\mathbf{b}$

图10 • 计算b，二元线性回归

- X^T 与X相乘得到格拉姆矩阵 $X^T X$ ，取逆后与 X^T 相乘得到广义逆，再与y相乘得到b

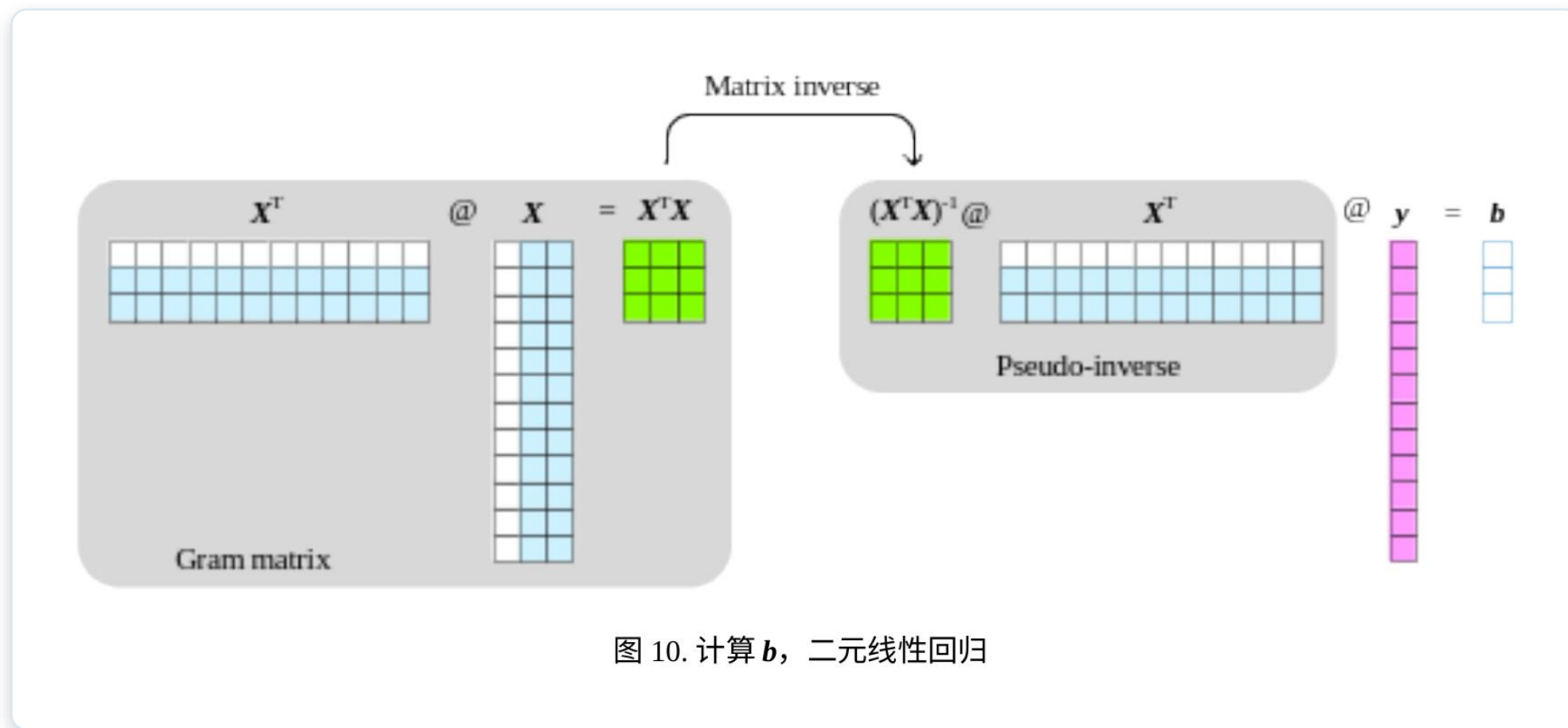


图 10. 计算 b ，二元线性回归

把 b 的解析式代入 $\hat{y}=Xb$ ，得到投影矩阵

$$\hat{y} = Xb = X(X^T X)^{-1} X^T y$$

- $X(X^T X)^{-1} X^T$ 就是 y 向 X 列向量张成平面的投影矩阵(projection matrix)

图11 • 计算 $\hat{y}$ ，二元线性回归

- X 、格拉姆矩阵的逆、 X^T 相乘得到投影矩阵，再与 y 相乘得到预测值 $\hat{y}$

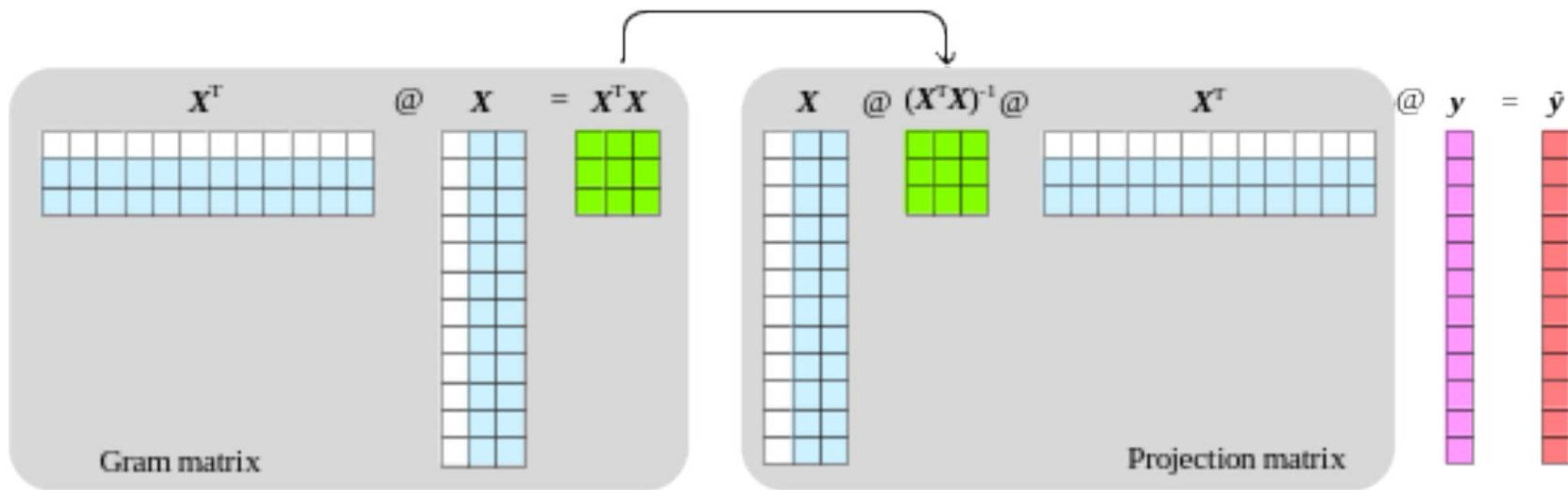


图 11. 计算 $\hat{y}$ ，二元线性回归

用Python完成二元线性回归求解

手写矩阵公式 $b = \text{inv}(X.T@X)@X.T@y$ 求解最优参数

- (a) 创建样本数据：
np.random.randn(n_samples,2)生成 x_1 、 x_2 ；y由 x_1 、 x_2 线性组合加噪声得到
- (b) 构造设计矩阵X：np.ones_like(y)造全1列，np.column_stack拼接得到 $n \times 3$ 的X
- (c) 计算参数向量 $b = \text{inv}(X^T X) @ X^T @ y$ ——最小二乘法的标准公式
- (d) 生成网格数据X_grid，前面加一列1，构造成跟训练数据一样的格式，用来做预测
- (e) 用计算得到的b，对网格点进行预测，得到y_pred，对应 $\hat{y} = Xb$
- (f) 绘制训练样本数据的三维散点图

```
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 创建数据
np.random.seed(0)
n_samples = 100
x_12 = np.random.randn(n_samples, 2)
y = -3*x_12[:,0] + 2*x_12[:,1] + 1 + 0.25*np.random.randn(n_samples)

# 构造设计矩阵，增加全1列向量
X = np.column_stack((np.ones_like(y), x_12))

## 创建线性回归模型并拟合数据
b = np.linalg.inv(X.T @ X) @ X.T @ y

## 拟合
# 生成回归平面的数据点
x1_grid, x2_grid = np.meshgrid(np.linspace(-3, 3, 10),
                                np.linspace(-3, 3, 10))
X_grid = np.column_stack((x1_grid.flatten(), x2_grid.flatten()))
X_grid = np.column_stack((np.ones_like(X_grid[:,0]), X_grid))

# 预测回归平面上的响应变量
y_pred = X_grid @ b
y_pred = y_pred.reshape(x1_grid.shape)

## 可视化
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
# 绘制三维样本散点
ax.scatter(x_12[:,0], x_12[:,1], y, marker = 'x')

# 绘制回归平面
ax.plot_wireframe(x1_grid, x2_grid, y_pred, colors = 'r')

ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
ax.set_zlabel('$y$')
ax.set_xlim([-3,3]); ax.set_ylim([-3,3])
ax.set_zlim([-15,15])
ax.set_proj_type('ortho'); ax.view_init(azim=-120, elev=30)
```

用Scikit-Learn自动完成模型训练

请大家自学LA_10_02_02.ipynb：代码使用Scikit-learn提供的LinearRegression工具，完成二元线性回归问题的求解

背后的数学原理与最小二乘法完全一致，结果也和LA_10_02_01.ipynb完全一致，请大家对比学习

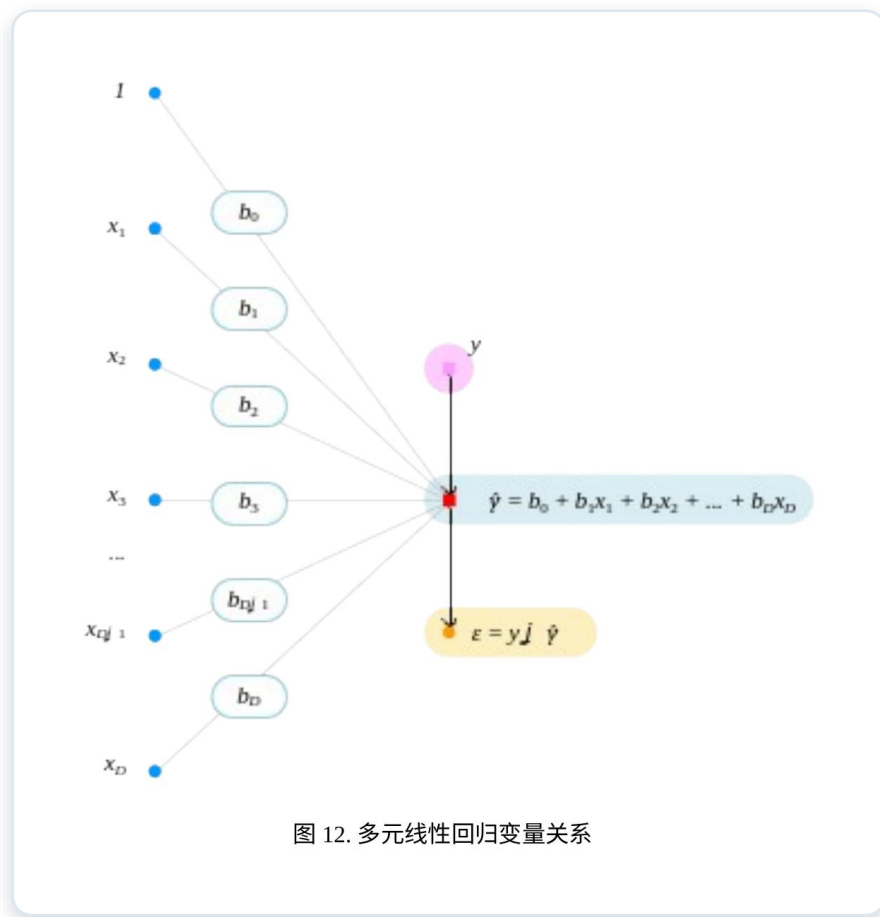
多元线性回归建立y与多个自变量之间的线性关系

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_Dx_D + \varepsilon$$

- b_0 为截距项, $b_1, b_2, \dots, b_D$ 代表自变量系数, ε 为残差项, D 为自变量个数
- 几何角度看, 多元线性回归得到一个超平面(hyperplane)——超平面是三维空间中平面的一种推广

图12 • 多元线性回归变量关系

- 常数1、自变量 x_1 到 x_D 分别乘以系数 b_0 到 b_D ，加和得到预测值 $\hat{y}$ ，与真实值 y 相减得到残差 ε



从向量空间角度看， $1, x_1, \dots, x_D$ 张成一个超平面

$$y = \hat{y} + \varepsilon, \quad \hat{y} = b_0 \mathbf{1} + b_1 x_1 + \dots + b_D x_D$$

- 向量 y 在 $\text{span}(1, x_1, x_2, \dots, x_D)$ 上投影结果为 $\hat{y}$ ，即 $\hat{y}$ 是 $1, x_1, \dots, x_D$ 的线性组合

图13 • 向量 y 向超平面 $\text{span}(1, x_1, x_2, \dots, x_D)$ 投影，多元线性回归

- 误差 $y - \hat{y}$ 垂直 $\text{span}(1, x_1, x_2, \dots, x_D)$, $\text{span}(x_1, \dots, x_D)$ 与 $\text{span}(\epsilon)$ 互为正交补

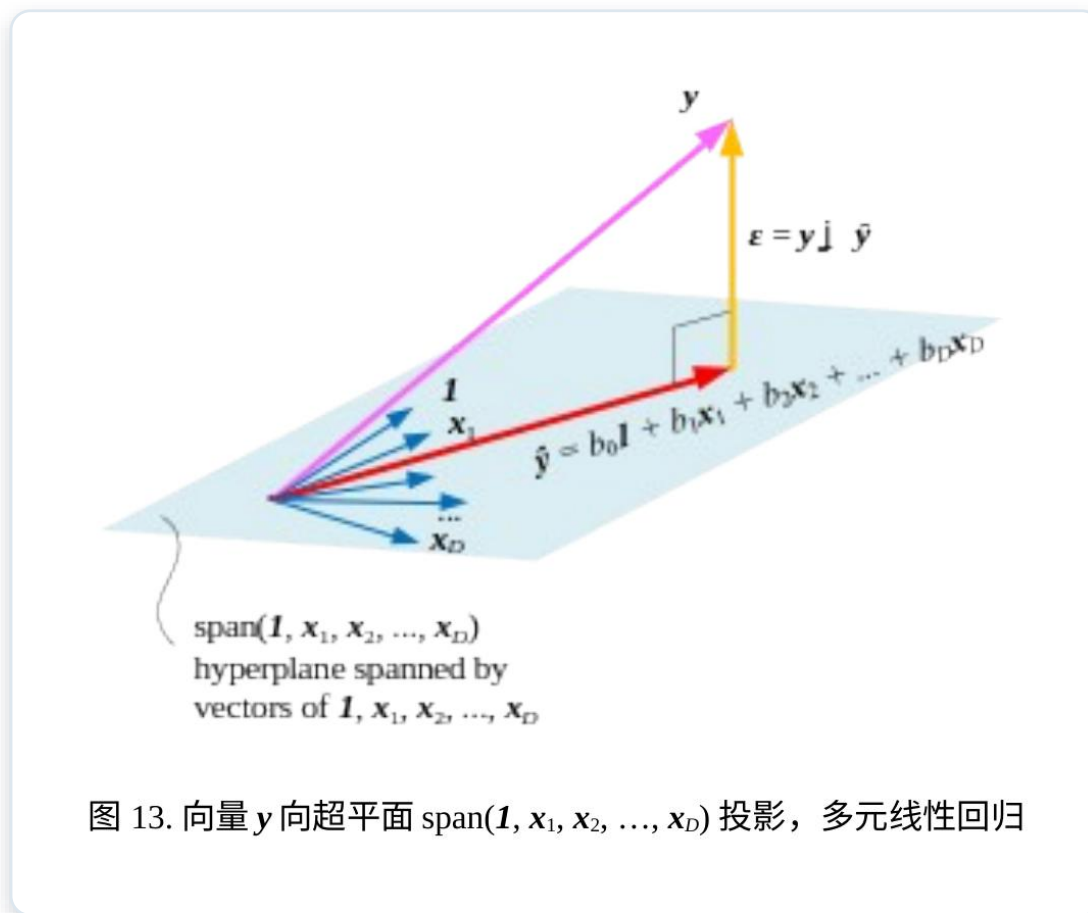


图 13. 向量 y 向超平面 $\text{span}(\mathbf{1}, x_1, x_2, \dots, x_D)$ 投影，多元线性回归

误差 $y-\hat{y}$ 分别垂直于 x_1 、 x_2 、...、 x_D ，写成设计矩阵

$$X_{n \times (D+1)} = [1 \ x_1 \ x_2 \ \dots \ x_D]$$

- 误差正交条件 $x_1^T(y-\hat{y})=0$, ..., $x_D^T(y-\hat{y})=0$, 合并写成矩阵形式 $X^T(y-\hat{y})=0$

图14 • 设计矩阵 X ，多元线性回归

- 把全1列向量与 x_1 到 x_D 共 D 列自变量数据横向拼接，得到 $n \times (D+1)$ 的设计矩阵 X

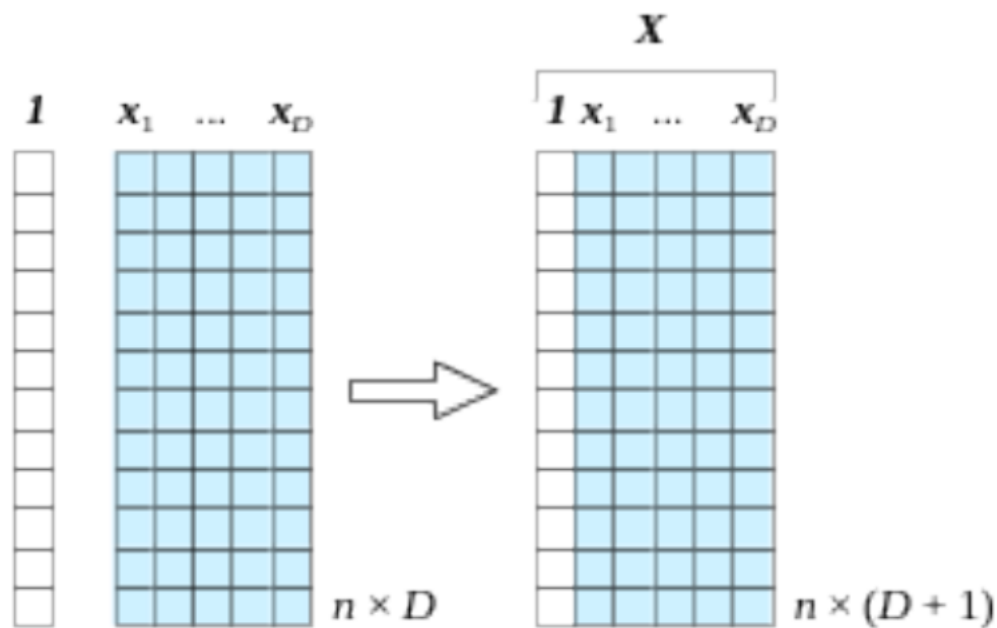


图 14. 设计矩阵 X ，多元线性回归

写成矩阵乘法形式

预测值构成的列向量 $\hat{y}$ ，通过矩阵乘法计算得到

$$\hat{y} = Xb$$

- b 是由 $D+1$ 个回归系数 b_0 、 b_1 、...、 b_D 堆叠而成的列向量

图15 • 矩阵运算得到 $\hat{y}$ ，多元线性回归

- 设计矩阵 X 与系数向量 b 做矩阵乘法，直接得到预测值列向量 $\hat{y}$

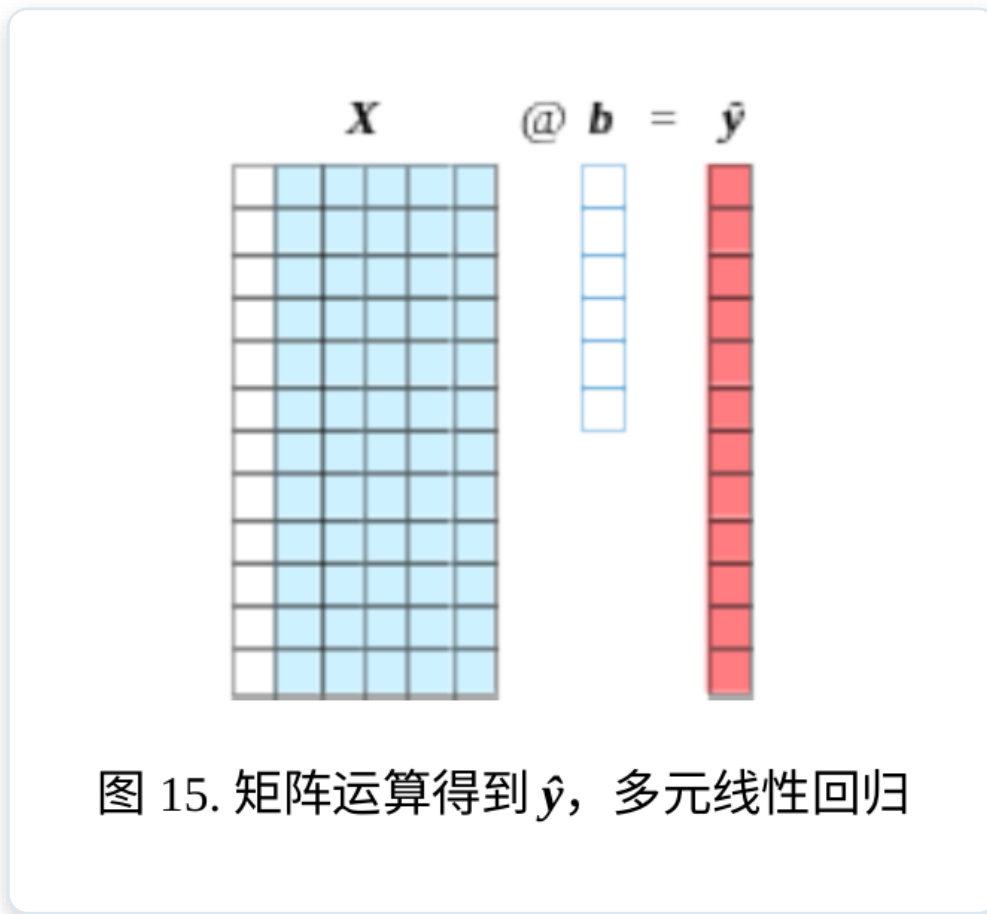


图 15. 矩阵运算得到 $\hat{y}$ ，多元线性回归

如果格拉姆矩阵可逆，b的解析解为

$$\mathbf{b} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- 只有格拉姆矩阵 $\mathbf{X}^T \mathbf{X}$ 满秩（ $\mathbf{X}$ 对应列满秩）， $\mathbf{X}^T \mathbf{X}$ 才可逆

图16 • 计算b，多元线性回归

- X^T 与 X 相乘得到格拉姆矩阵 $X^T X$ ，取逆后与 X^T 相乘得到广义逆，再与 y 相乘得到 b

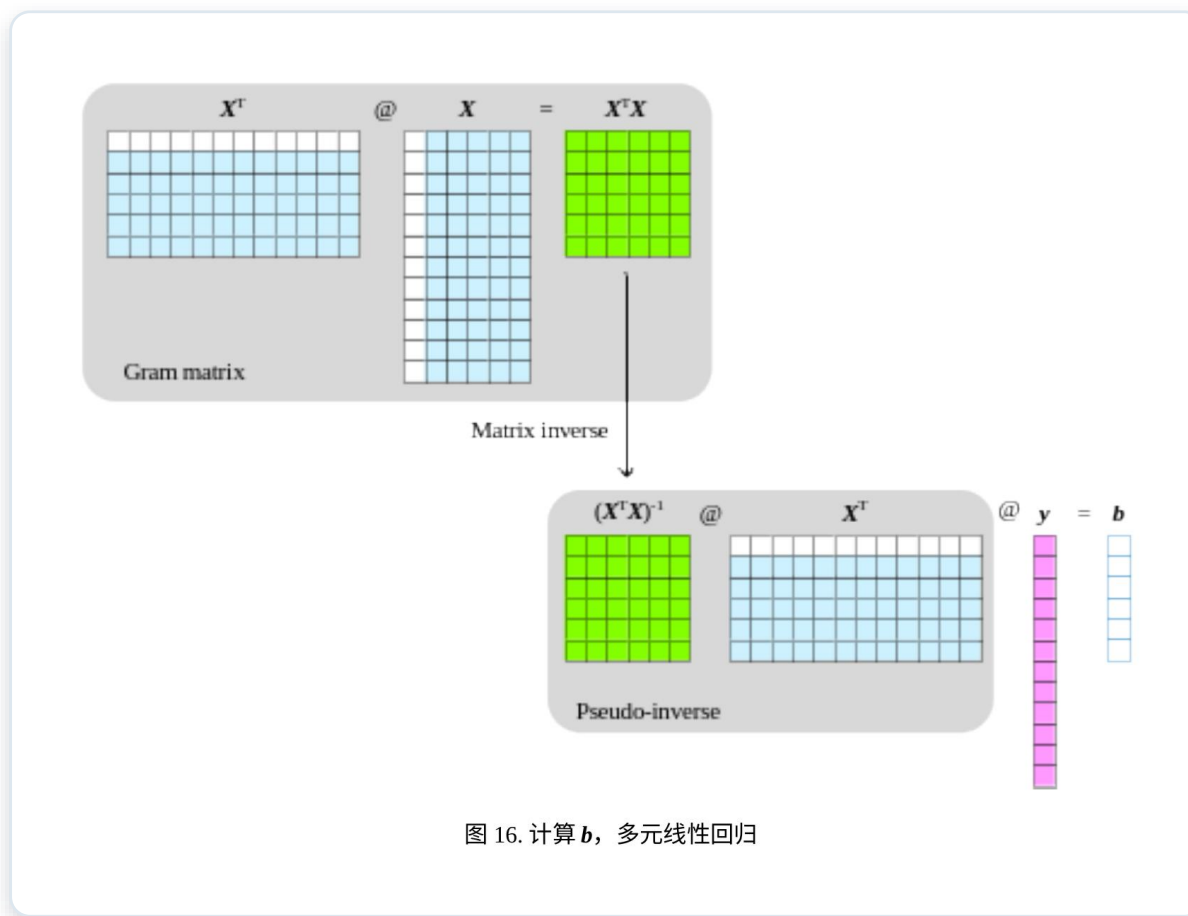


图 16. 计算 b ，多元线性回归

求解 $\mathbf{b}$ ，本质上是求解一个超定方程组

超定方程组 $\mathbf{X}\mathbf{b}=\mathbf{y}$ ：方程个数（样本数 n ）多于未知数个数（ $D+1$ 个回归系数）
等式左右乘 $\mathbf{X}^T$ ，得到 $\mathbf{X}^T\mathbf{X}\mathbf{b}=\mathbf{X}^T\mathbf{y}$ ，其中 $\mathbf{X}^T\mathbf{X}$ 便是 $\mathbf{X}$ 的格拉姆矩阵

左右乘 $\mathbf{X}^T\mathbf{X}$ 的逆，得到 $\mathbf{b}=(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ ——这是从超定方程组视角求解 $\mathbf{b}$

对设计矩阵X进行完全型QR分解

$$X = QR, \quad Q = [Q_1 \ Q_2], \quad R = [R_1; \mathbf{0}]$$

- Q为正交矩阵，是个方阵；整理分块乘法之后，得到经济型QR分解 $X = Q_1 R_1$

R_1 是上三角矩阵，计算逆更方便

矩阵乘法可以写成 $\hat{y} = Q_1 R_1 b$

上式左右乘半正交矩阵 Q_1^T ，由于 $Q_1^T Q_1 = I$ ，得到 $Q_1^T \hat{y} = R_1 b$

R_1 是上三角矩阵，计算逆比对格拉姆矩阵求逆更方便——这便是QR分解在求解方程组中的优势所在

拟合值向量 $\hat{\mathbf{y}}$ ，可以写成投影矩阵与 $\mathbf{y}$ 的乘积

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{b} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

- $\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ 就是 $\mathbf{y}$ 向 $\mathbf{X}$ 列向量张成超平面的投影矩阵，本书前文称其为帽子矩阵(hat matrix)

图17 • 计算 $\hat{y}$ ，多元线性回归

- X 、格拉姆矩阵的逆、 X^T 相乘得到投影矩阵，再与 y 相乘得到预测值 $\hat{y}$

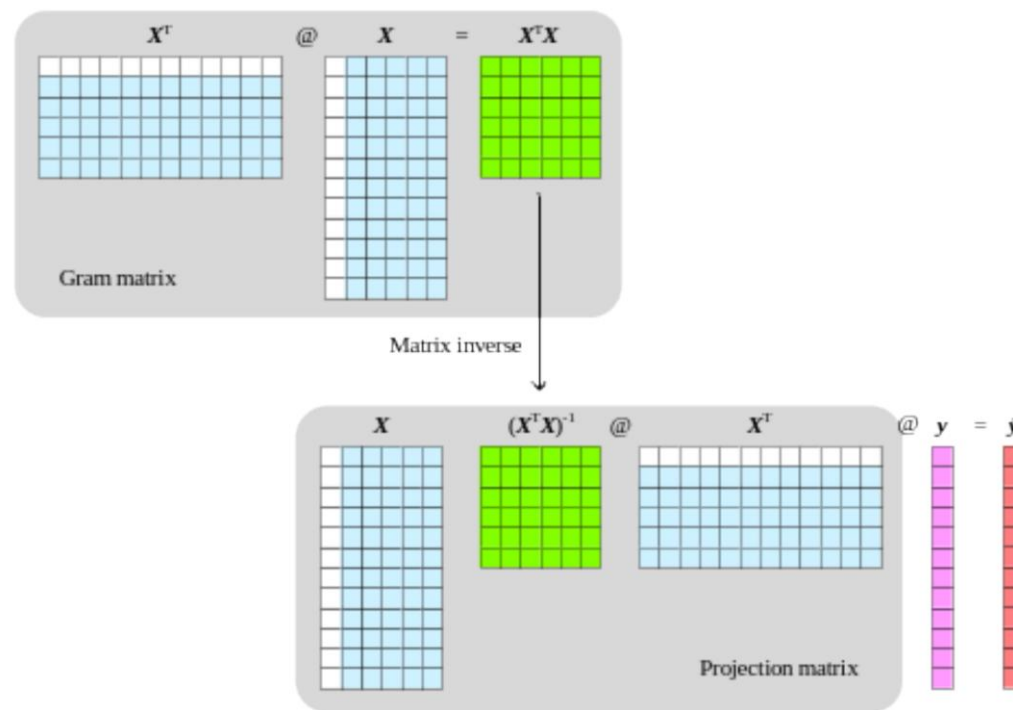


图 17. 计算 $\hat{y}$ ，多元线性回归

残差向量 ϵ 同样可以写成一个投影矩阵与 y 的乘积

根据 $\epsilon = y - \hat{y}$ ，残差列向量可以写成 $\epsilon = (I - X(X^T X)^{-1} X^T)y$

$\text{span}(\epsilon)$ 是 $\text{span}(1, x_1, \dots, x_D)$ 的正交补；换个角度看， ϵ 就是该超平面的法向量

$I - X(X^T X)^{-1} X^T$ 也是一个投影矩阵，把 y 投影到"残差"空间，得到残差向量 ϵ

把 $\hat{y}$ 、 ϵ 相加： $\hat{y} + \epsilon = y$ ，向量 y 正交分解为估计值向量 $\hat{y}$ 、误差向量 ϵ

从二元到多元回归，我们看到了什么？

二元线性回归对应三维空间的一个平面，多元线性回归对应一个超平面

b_0 控制平面高度， b_1 、...、 b_D 控制沿各自变量方向的斜率

预测值 $\hat{y}$ 是真实值 y 在 $\text{span}(1, x_1, \dots, x_D)$ 上的正交投影，残差 ε 与该空间正交

设计矩阵 $X = [1, x_1, \dots, x_D]$ ，格拉姆矩阵 $X^T X$ 可逆时， $b = (X^T X)^{-1} X^T y$

投影矩阵 $X(X^T X)^{-1} X^T$ 把 y 映射为 $\hat{y}$ ； $I - X(X^T X)^{-1} X^T$ 把 y 映射为 ε ，两者正交互补

QR分解把 $X = Q_1 R_1$ ，用上三角矩阵 R_1 求逆，是求解回归问题的另一条路径

本节关键术语

二元/多元线性回归 *Multiple Linear Regression* —— 因变量随两个或多个自变量线性变化的模型

截距、斜率系数 *Intercept b_0 / Slope Coefficients $b_1, \dots, b_D$*

设计矩阵 *Design Matrix X* —— 第一列为全1，其余列为各自变量取值

广义逆 *Generalized Inverse / Pseudo-Inverse* —— $(X^T X)^{-1} X^T$

QR分解 *QR Decomposition* —— $X=QR$ ，用正交矩阵与上三角矩阵求解回归问题

超平面 *Hyperplane* —— 三维空间中平面在更高维度的推广

残差 *Residual ε* —— 真实值与预测值之差， $\varepsilon=y-\hat{y}$

格拉姆矩阵 *Gram Matrix* —— $X^T X$ ，回归中用来求解最优参数

超定方程组 *Overdetermined System* —— 方程个数多于未知数个数的方程组 $Xb=y$

投影矩阵、帽子矩阵 *Projection Matrix / Hat Matrix*
 $H=X(X^T X)^{-1} X^T$ ，满足 $\hat{y}=Hy$

延伸阅读

回顾 · 10.1 从投影到回归

本节内容建立在一元线性回归、正交投影的基础上，建议平行阅读

延伸阅读 · 10.3 多项式回归

下一节将介绍非线性关系如何转化为线性回归问题

延伸阅读 · 《概率统计不难》

显著性检验、置信区间、拟合优度、多重共线性

开源仓库

全书代码与图像素材：<https://github.com/Visualize-ML>

习题 Q1 – Q9

Q1 已知代码产生多元回归自变量、因变量数据，请用Python编程计算 b 、 $\hat{y}$ 、 ε 。

Q3 为什么二元线性回归对应三维空间的一个平面，而不是一条直线？

Q5 设计矩阵 X 的第一列为什么必须是全1列向量？如果去掉这一列，回归模型会发生什么变化？

Q7 证明：残差向量 ε 与设计矩阵 X 的每一列都正交。

Q9 写出二元线性回归中， b_1 、 b_2 分别为0时，回归平面各自退化成什么形状？

Q2 用sklearn.linear_model模块中的LinearRegression函数求解Q1中的多元线性回归，并比较结果。

Q4 写出二元线性回归 $\hat{y}=b_0+b_1x_1+b_2x_2$ 中， $b_0=0$ 时平面必然经过哪个点？为什么？

Q6 什么条件下格拉姆矩阵 $X^T X$ 不可逆？举一个自变量之间线性相关的例子。

Q8 投影矩阵 $H=X(X^T X)^{-1}X^T$ 满足 $H^2=H$ 吗？请用矩阵运算验证。

习题 Q10 – Q17

Q10 用numpy构造一个5元线性回归的设计矩阵X，并计算其格拉姆矩阵 $X^T X$ 。

Q12 用numpy.linalg.qr对设计矩阵X做经济型QR分解，并用 Q_1 、 R_1 求解b。

Q14 写Python函数hat_matrix(X)，返回投影矩阵 $H = X(X^T X)^{-1} X^T$ 。

Q16 用matplotlib可视化三元线性回归的残差平方和随两个系数变化的等高线图。

Q11 写Python函数fit_ols(X, y)，返回最小二乘估计的系数向量b。

Q13 比较用格拉姆矩阵求逆与用QR分解求解b，两种方法结果是否一致。

Q15 验证残差向量 $\varepsilon = (I - H)y$ 与预测值向量 $\hat{y} = Hy$ 是否正交（内积是否为0）。

Q17 写Python函数r_squared(y, y_hat)，计算拟合优度 R^2 。

习题 Q18 – Q25

Q18 探索：当自变量个数 D 远大于样本数 n 时， $X^T X$ 是否还可逆？会出现什么问题？

Q20 写Python函数`standardize(X)`，对设计矩阵每一列做标准化，再对比回归系数的变化。

Q22 用代码验证： $Q_1^T Q_1$ 是否等于单位矩阵 I ，这一性质在QR分解求解回归时起什么作用？

Q24 探索多元线性回归在实际数据集（如房价预测）中的应用：选择2-3个特征，拟合模型并可视化。

Q19 用代码生成一组存在多重共线性的自变量数据，观察 $X^T X$ 条件数的变化。

Q21 探索：去掉设计矩阵中的某一个自变量列，重新拟合，比较 R^2 和系数的变化。

Q23 综合练习：写Python函数`predict(X_new, b)`，对新样本数据批量做预测。

Q25 预习思考：下一节将介绍多项式回归(10.3)，多项式回归的设计矩阵与本节有什么联系与区别？

10个可以和AI互动的Prompt

- 1 请解释为什么二元线性回归的几何图像是一个平面，而多元线性回归是超平面。
- 2 请解释格拉姆矩阵 $X^T X$ 在最小二乘回归中扮演的角色，为什么它需要满秩才能求逆。
- 3 请用Python演示：如何用numpy从零实现多元线性回归的最小二乘求解。
- 4 请解释广义逆 $(X^T X)^{-1} X^T$ 与投影矩阵 $X(X^T X)^{-1} X^T$ 之间的关系。
- 5 请解释QR分解求解回归问题相比直接求逆格拉姆矩阵的数值优势。

10个可以和AI互动的Prompt

6 请解释为什么残差向量 ϵ 与设计矩阵 X 的列空间正交，这一性质如何用来求解 b 。

7 请解释帽子矩阵(hat matrix)这一名称的由来，以及它满足 $H^2=H$ 的几何含义。

8 请解释多重共线性对多元线性回归系数估计的影响，以及常见的应对方法。

9 请解释残差平方和 $\|\epsilon\|^2$ 与拟合优度 R^2 之间的数学关系。

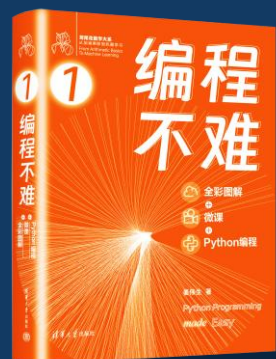
10 请用Python可视化：同一份数据分别用一元、二元、多元线性回归拟合的效果对比。

数学

数学基础

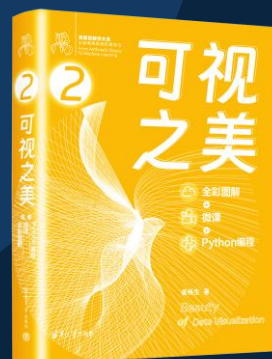
线性代数

概率统计

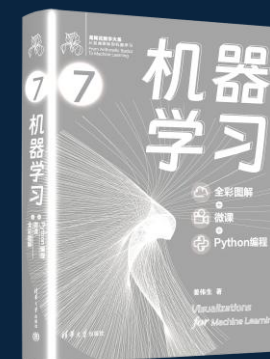
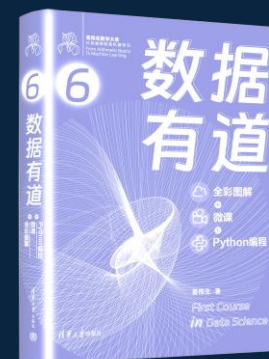
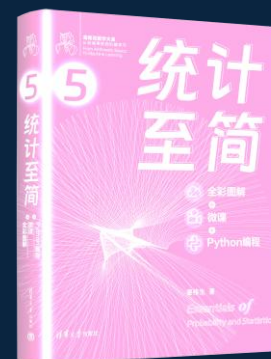
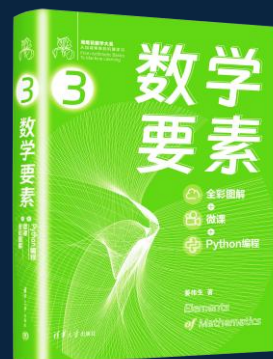


Python编程

工具



数据可视化



数据分析
图和网络

回归、降维
分类、聚类

实践

